Networks & Distributed Programming

The Application Layer

Lecture 3



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



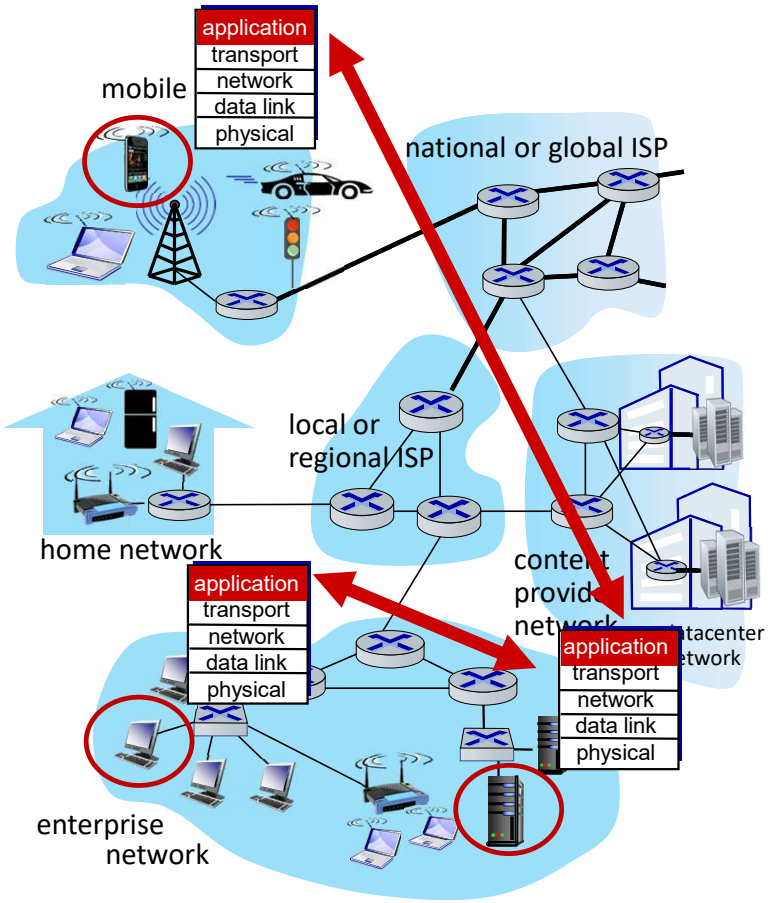
DIETI

UNINA

Application Layer
Network Applications

- A **network application** is composed by multiple programs that run on different end-systems (hosts or endpoints) and communicate with each other over the network.

- *Example*: a Web application includes two distinct programs:
  - the **browser program** running in the user's host (desktop, laptop, tablet, smartphone, and so on).
  - the **Web server program** running in the Web server host.



Application Layer
Network Applications

We have the whole stack.
Now we open the layer at the top, the one the user sees.

- An application does not move data: it hands data down and gets it back at the other end
- What the transport layer offers upward is a reliable data stream
- **Two applications** on two hosts behave as if they **talked directly**, provided both follow the **same protocol**

So what is left for the application programmer to do?

Everything above that stream: which messages exist, what they mean, and who speaks first.

Application Layer

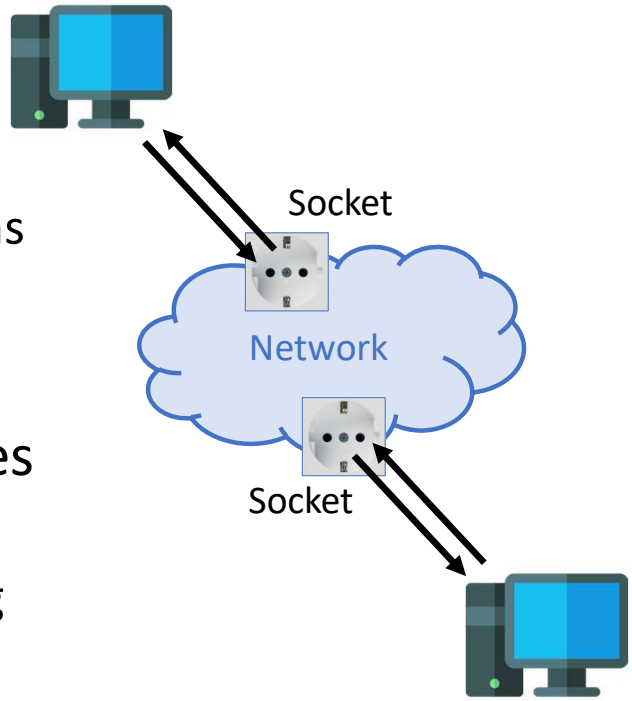
Some Network Applications

- The **applications** are the programs that runs on the **devices** and allow users to access to **services**.
  - social networking
  - Web
  - text messaging
  - e-mail
  - multi-user network games
  - streaming stored video (YouTube, Netflix)
  - P2P file sharing
  - voice over IP (e.g., Skype)
  - real-time video conferencing
  - Internet search
  - remote login
  - …

Application Layer

Creating Network Applications

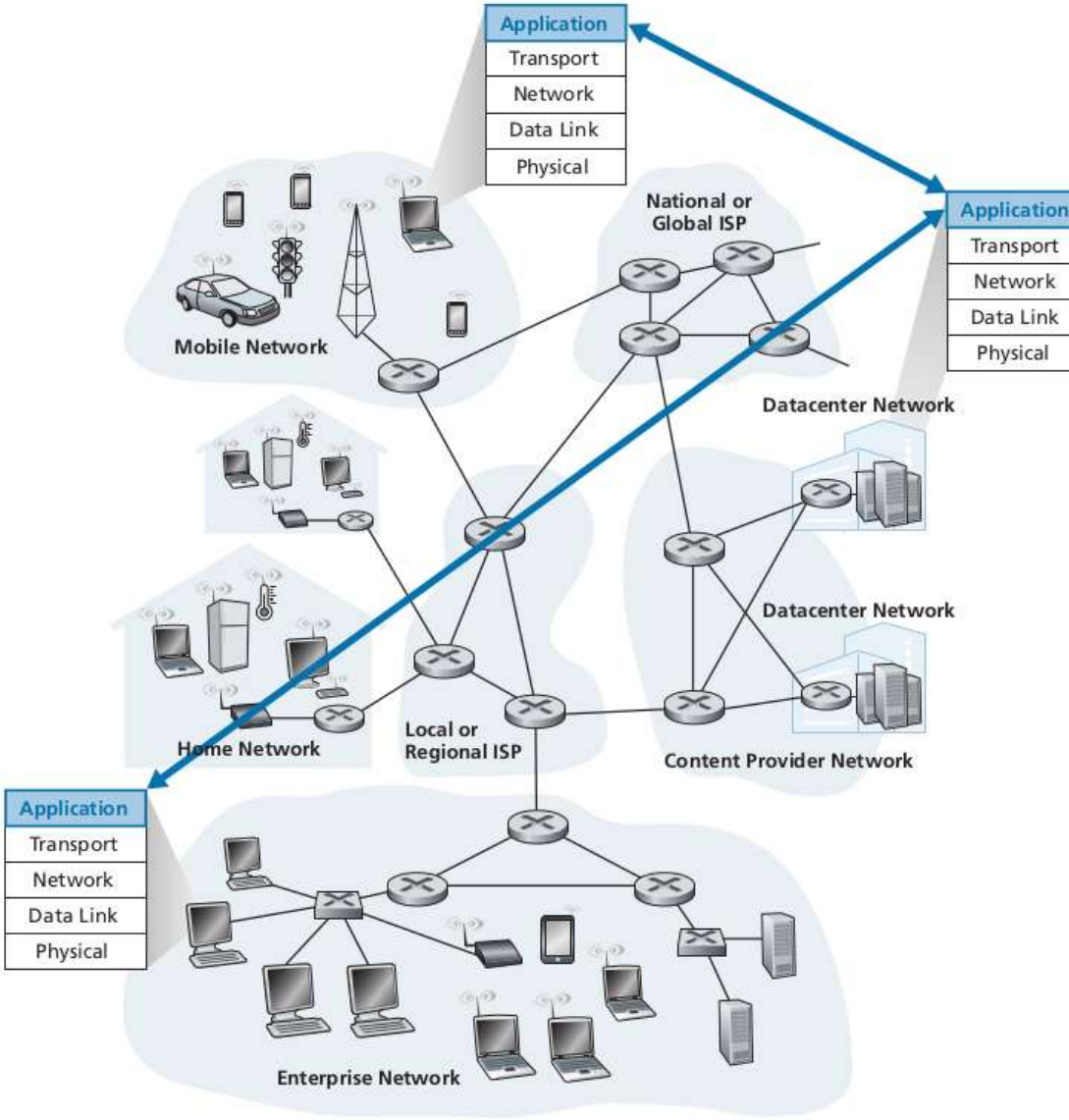
- Creating network applications means to write programs that:
  - **Run on different end systems**, perhaps using different languages or OS.
  - **Communicate over network** (e.g., via sockets) by means of a specific protocol.



- No need to write software for network-core devices
  - Network devices **do not run user applications**.
  - There are **specific libraries** (e.g., sockets) implementing network functionalities.

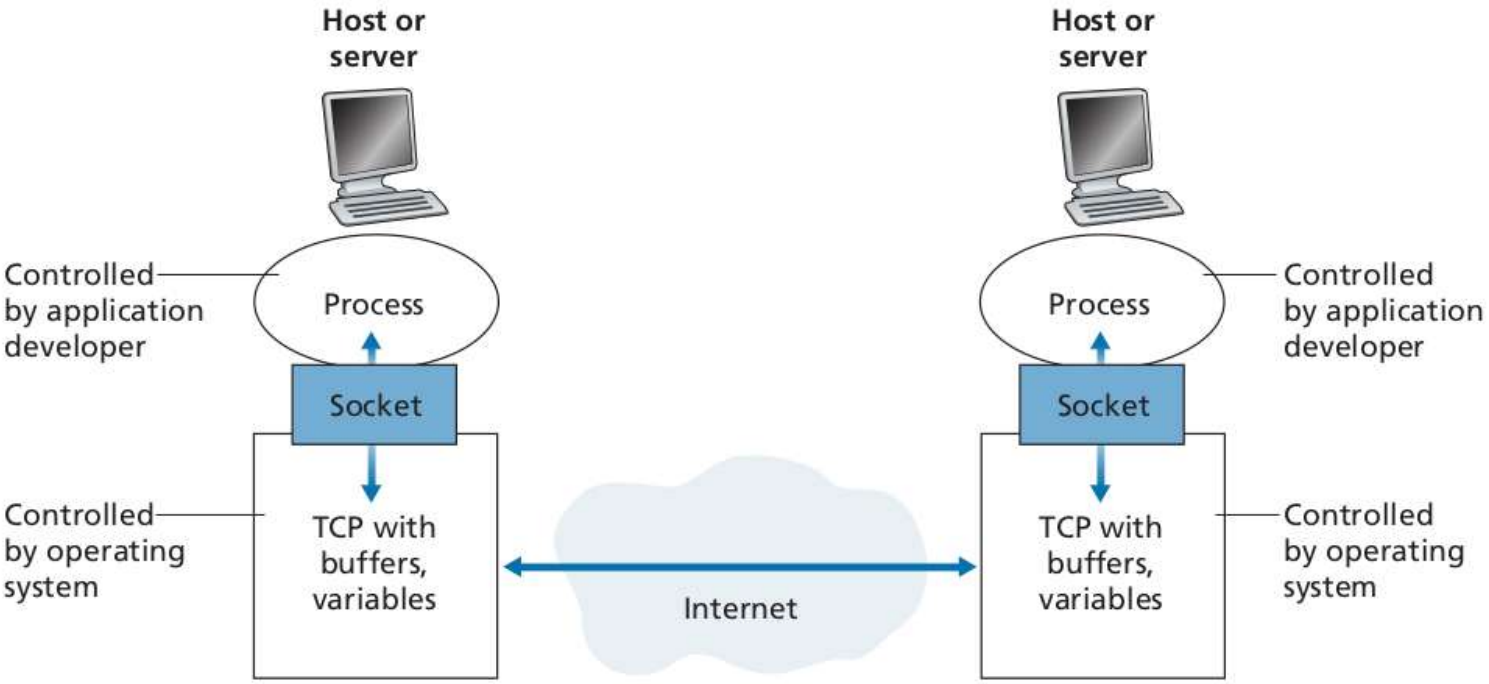
Application Layer

Creating Network Applications



Application Layer

Creating Network Applications



- The communication between processes occurs via sockets.
- Applications use the transport layer's services (reliable data stream)

Application Layer
Network Applications

- A **(network) application architecture** is designed by the *application developer* and determines how the application is structured over the various end systems:
  - **Client-server**: the nodes are heterogeneous, there is an always-on host (server), which provides services to be requests from many other hosts (clients).
  - **Peer-to-peer (P2P)**: the nodes are (ideally) homogeneous, the application exploits direct communication between pairs of intermittently connected hosts (peers).

Client-server





Peer-to-peer

Application Layer

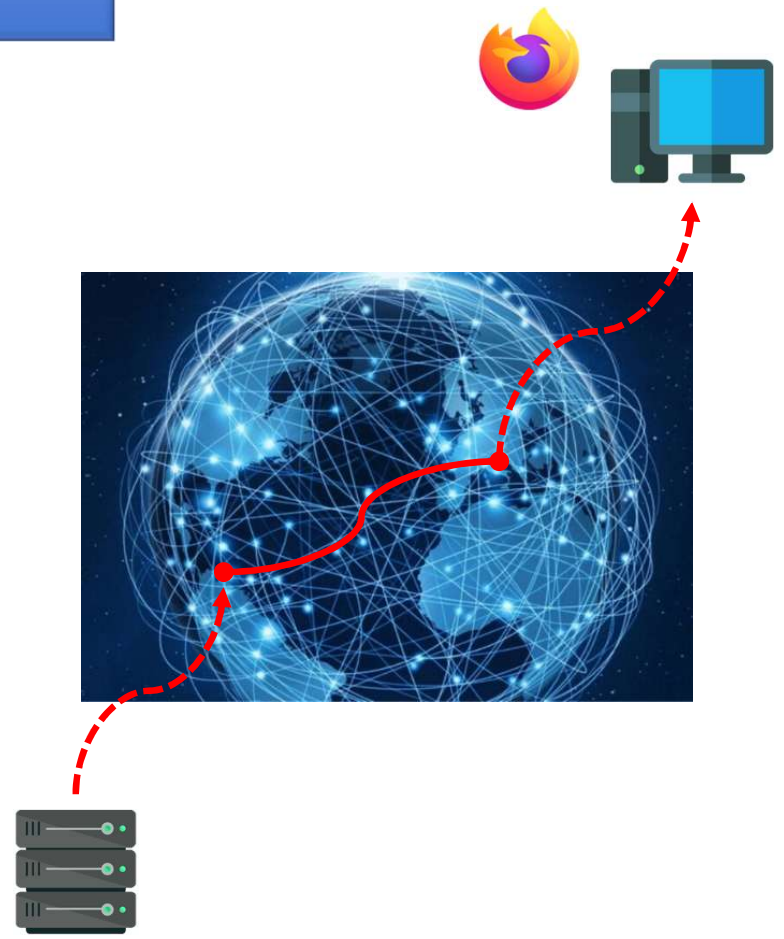
Client-server Applications

- In a client-server architecture, **clients do not directly communicate** with each other.
- The server has a **fixed, well-known address** (hostname or IP address) so a client can always contact the server by sending a packet (request) to it.
- In a client-server architecture **multiple servers** are often involved to keep up with all the requests from clients. The **client is typically unaware** of that and perceives them as a single server.
- **Multiple servers** can be:
  - Grouped into **data centers** containing a huge number of servers in a specific location.
    - Servers must be powered, maintained, and well connected.
  - Scattered as **distributed servers** all around the world.
    - Servers must be interconnected and coordinated.
  - Organized in **distributed data centers**.

Application Layer

Client-server Applications

- A **web application** is a typical client-server example:
  - There is an **always-on Web server** receiving requests from the browsers on the client hosts.
  - When a Web server receives a **request** for an object from a client host, it **responds** by sending the requested object to the client host.
  - The web server is **reachable** by the hosts.



Application Layer
Peer-to-peer Applications

- It is often used for **traffic-intensive applications**

- There is no single server with fixed address
  - Each client has its own address

- There is minimal, or no, reliance on always-on servers:
  - the peer are the users' own machines, connected on and off
  - Each peer generates load by asking for a file, and adds capacity by serving it to others
  - **Capacity grows together with demand**. This is self-scalability, and no client-server design has it.
  - It is also **cheap:** no server infrastructure to build, no server bandwidth to pay.
  - The price is **security, performance and reliability**

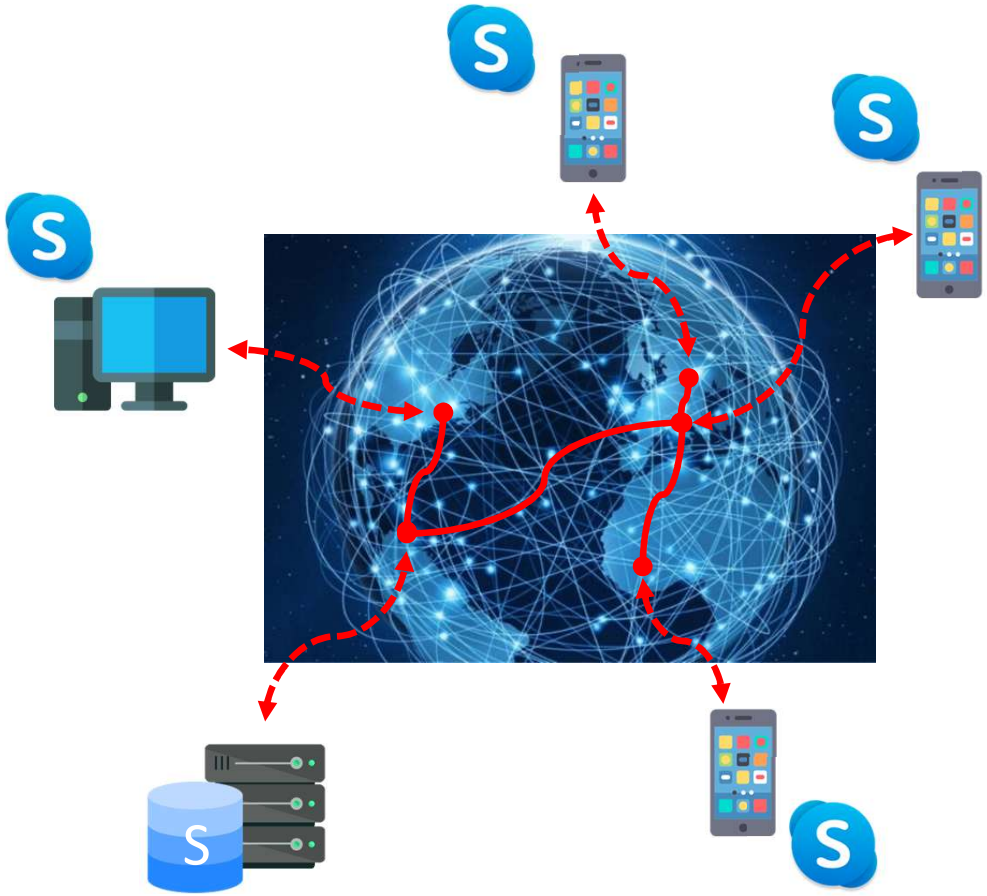
Application Layer
Peer-to-peer Applications

- Pure P2P applications are rare:
  - P2P does not necessarily mean “no servers”. Servers may be used for discovery and coordination, while the actual data is exchanged directly between peers.
  - Many applications have **hybrid** **architectures**, combining both client-server and P2P elements.
  - E.g., for many instant messaging applications, **servers are used to track the addresses of users**, but user-to-user messages are sent directly between user hosts (without passing through intermediate servers).

Application Layer

Peer-to-peer Applications

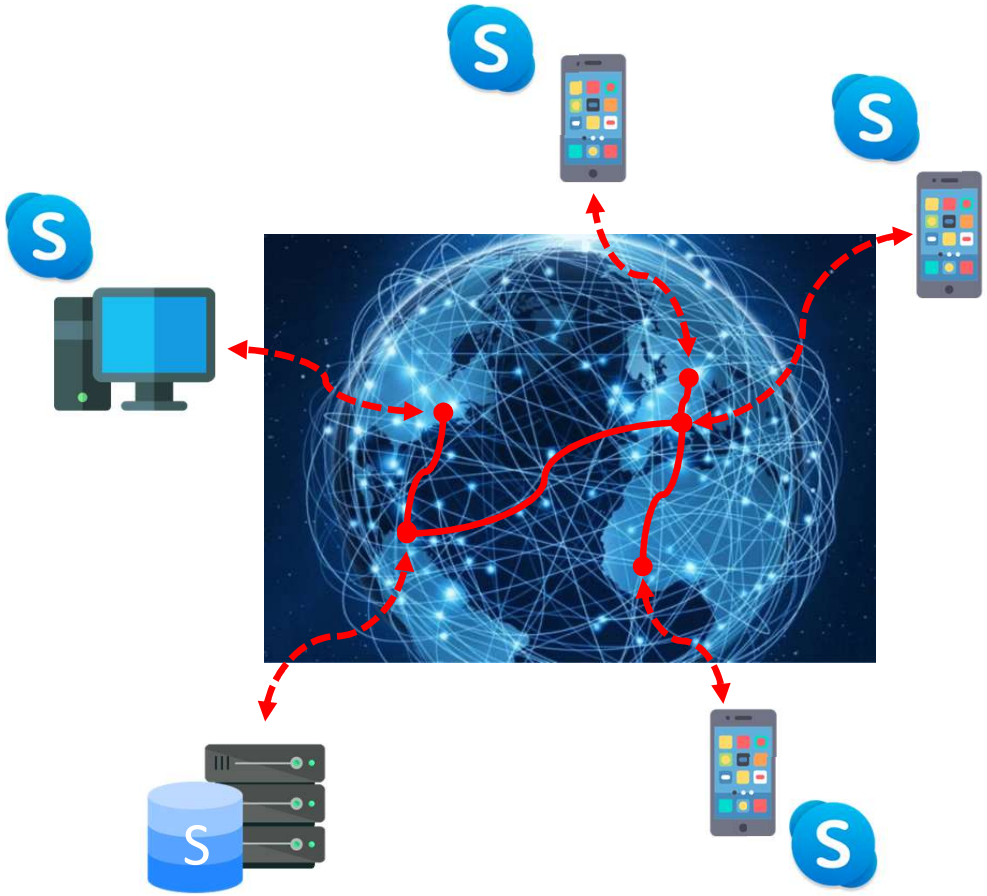
- Typical P2P applications are internet **telephony and video conference** or **file-sharing** (e.g., BitTorrent):
  - Servers are used to **track the IP addresses of users**
  - but user-to-user messages/requests are sent directly between user hosts (without passing through intermediate servers).
  - In file-sharing applications the server can also **trace all available files** in order to speed up the search.



Application Layer

Peer-to-peer Applications

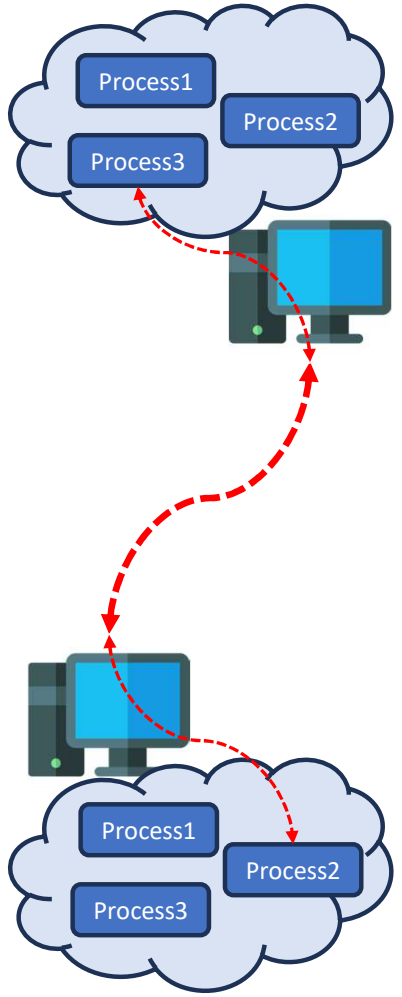
- Many P2P applications use a hybrid architecture:
  - In messaging or real-time communication, servers may provide user discovery, presence, and signaling; communication may then occur directly between peers when possible
  - In file-sharing systems, centralized services may provide peer discovery or content indexing to simplify searching



Application Layer

Communication between Processes

- In a network application there are **processes** running on different machines (potentially running on **different operating systems**) that communicate through the network.
  - In *web applications* the process of a client's browser exchanges messages with the process of the web server.
  - In *P2P file-sharing* a file is transferred from a process in one peer to a process in another peer.
- Between a pair of communicating processes there is typically a **client process** and a **server process**:
  - In W*eb applications* a browser's process is a client, while the server's process is a server.
  - In *P2P file-sharing* the peer that is downloading can be seen as a client, while the peer that is uploading as a server.
- In a P2P application processes may "change role".

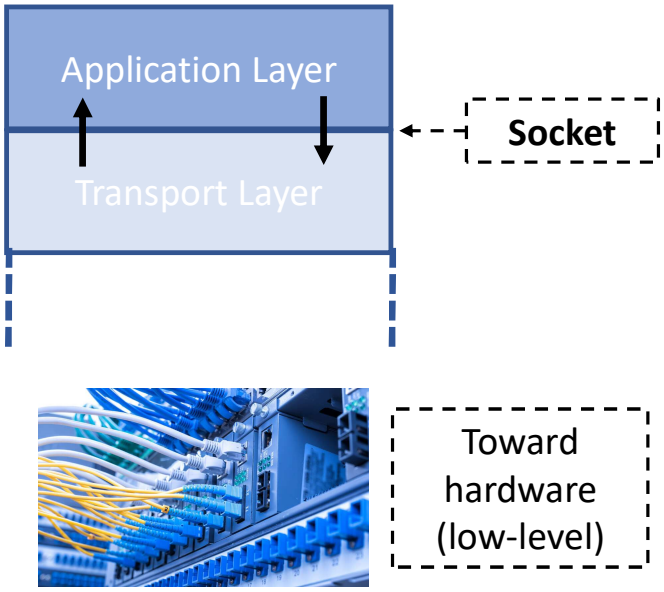


Application Layer

Communication between Processes

- Most network applications consist of **pairs of communicating processes** sending/receiving messages to/from each other through the underlying network.

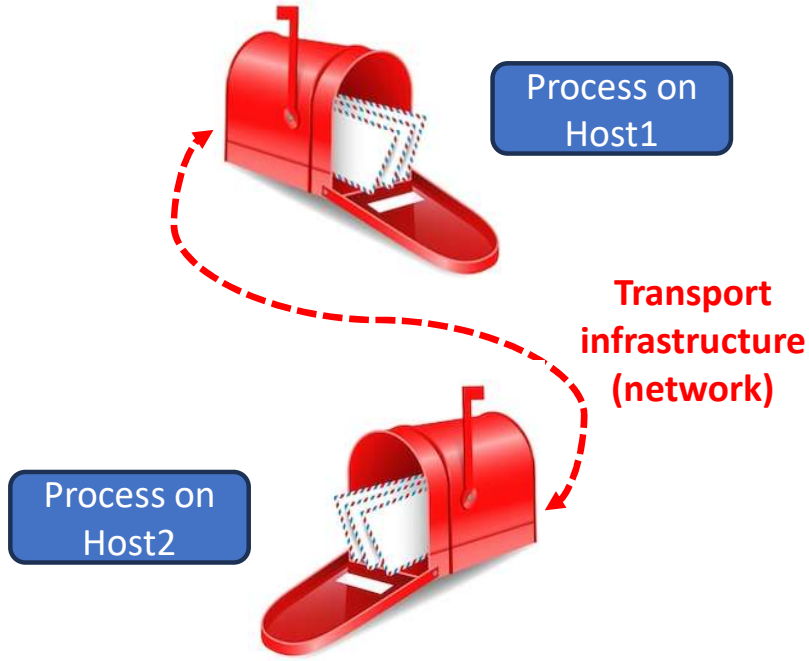
- Following the layered model, a **socket** is the **software interface** between application and transport layers and allows processes to send/receive messages over the network.



Application Layer

Communication between Processes

- A typical analogy is to consider sockets as **mailboxes**:
  - When a process wants to send a message to another process on another host, it puts the message **into a mailbox**.
  - This sending process **assumes** that there is a **transportation infrastructure** on the other side of its door that will transport the message to the mailbox of the destination process.
  - Once the message arrives at the destination host, the message passes through the **receiving process's mailbox** (socket).



Application Layer

Communication between Processes

- **Sockets** are used as **Application Programming Interface** (API) between the application and the network.

- The application developer has control of everything on the application-layer side of the socket but **has little control of the transport-layer** side of the socket.

- The only control that the application developer has on the transport-layer side is:
  1. The **choice** of transport protocol (TCP/UDP).
  2. The ability to set a few **transport-layer parameters** (e.g., maximum buffer, maximum segment size, etc.). Everything on the application side of the socket is yours. On the transport side you have almost nothing.
  3. That is all. From then on the application is built on the services that protocol provides.

- Once the application developer chooses a transport protocol (if a choice is available), the application is built assuming as given the transport-layer services provided by that protocol.

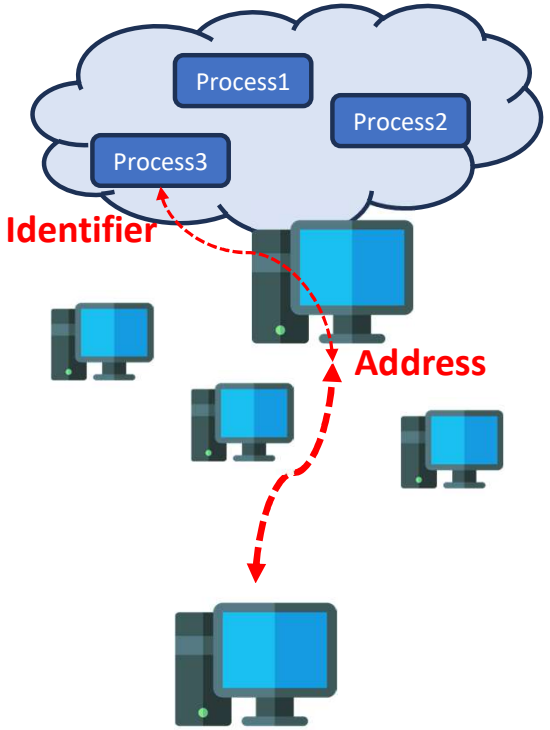
Application Layer

Communication between Processes

- Following the postal mail analogy, **processes need an address** to send messages.

- Since **multiple network applications** can be running on a **single host**, to identify the receiving process, two pieces of information needed:
  1. An **address** of the host (to find the right host on the network).
  2. An **identifier** of the receiving process (to find the right process in the host).

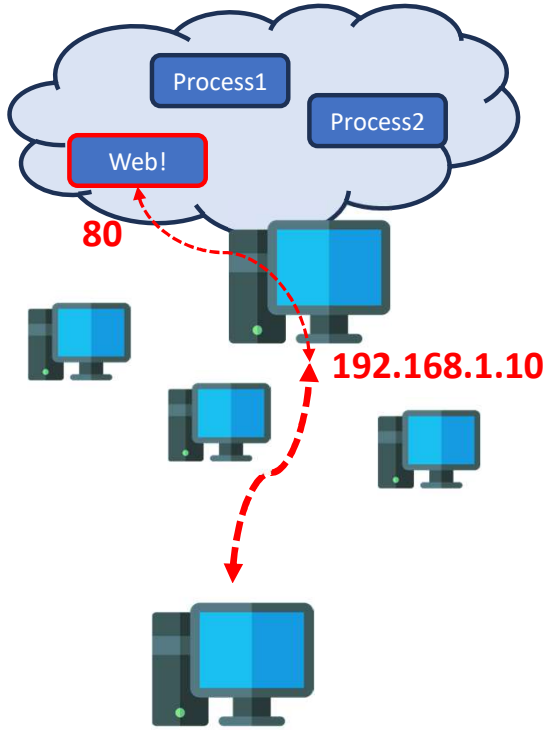
- The **address** works at the **network** level (routing) while **identifier** works at **transport** level.



Application Layer

Communication between Processes

- In the Internet, a host is identified by an **IP address** (32-bit quantity) while the process is identified by a **port number**.



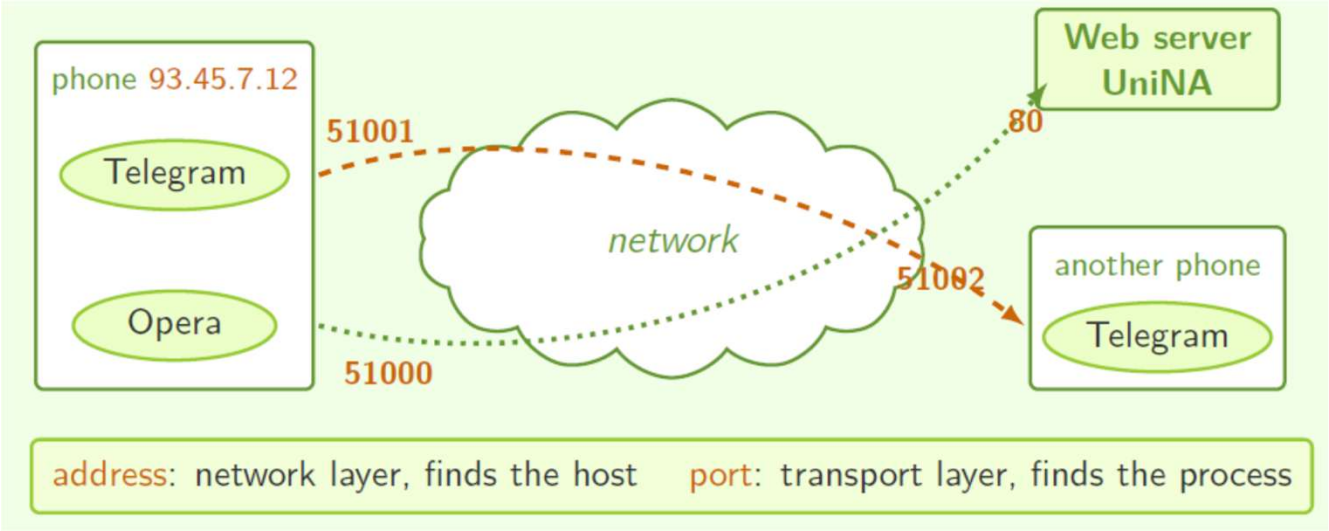
- Popular applications have been conventionally assigned to **specific port numbers**.
  - For example, a Web server is identified by port number 80, a mail server process (using the SMTP protocol) is identified by port number 25.

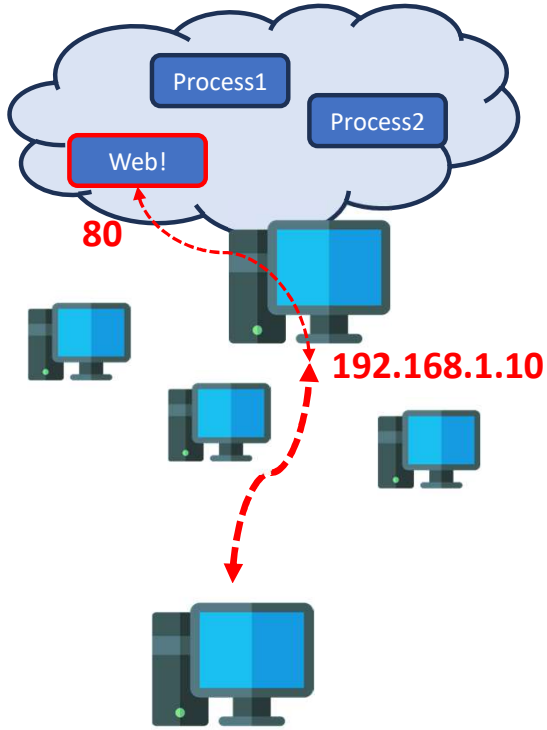
- IANA (Internet Assigned Numbers Authority)
  - A list of well-known port numbers (and protocols) can be found here: https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml.

Application Layer

Communication between Processes

- A phone with the messaging app and the browser both open, talking to two different destinations at the same time
- Both flows leave from the same address. What keeps them apart, at either end, is the port





Application Layer

Communication between Processes

- A laptop is on Ethernet and on Wi-Fi at once. How many addresses does it have?

Application Layer

Communication between Processes

- A laptop is on Ethernet and on Wi-Fi at once. How many addresses does it have?
  - Two, one per interface

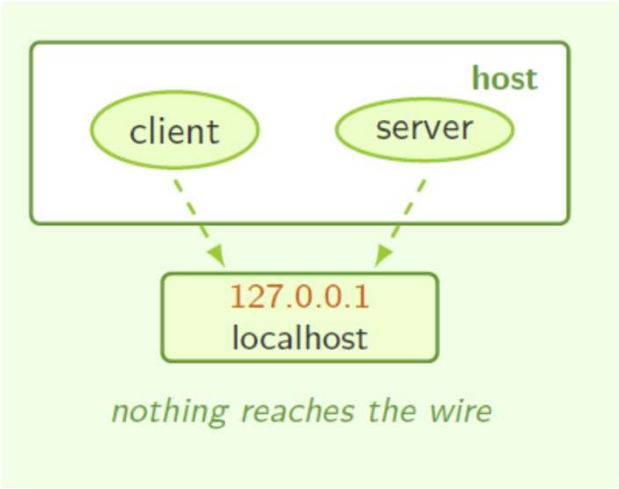
- An IP address does not name a machine: it names one attachment point to one network:
  - For the network layer that host counts as two distinct hosts, reached along two different paths.
  - Unplug the cable and one of the two addresses stops answering, while the other keeps working.

We will come back to this when addresses stop belonging to machines and start belonging to networks.

Application Layer

IPv4, and the address that means “here”

- IPv4 address
  - 32 bits, written as four decimal numbers between 0 and 255 separated by dots — one number per byte.
  - **127.0.0.1** is special: it does not name a remote machine, it names this one. Two local processes talk through it exactly as they would across the network, and nothing ever leaves the host.
  - It is how you will test a client against your own server, on one machine, before there is a second one.



Application Layer

IPv6, and the address that means “here”

- IPv6 addresses are 128 bits long and are written in hexadecimal notation.
- The **IPv6 loopback address is ::1**, the equivalent of IPv4 127.0.0.1.
- The name localhost may resolve to 127.0.0.1, ::1, or both.

Application Layer

What happens with localhost?

- When two applications communicate through localhost, data still passes through the local TCP/IP stack:

Application → Transport (TCP/UDP) → Network (IP) → loopback interface

- No Ethernet/Wi-Fi frame is sent
- No MAC address is needed
- No physical signal leaves the host
- The packet is internally returned to the receiving process

Application Layer

IPv4, and the address that means “here”

- Nobody types 143.225.161.30. People type www.unina.it, and the two designate the same Web server.
- The **name is for people**: readable, stable, and it survives a change of machine underneath.
- The address is what the network layer works on: it is what routing needs in order to deliver.
- **DNS:** the Domain Name System translates one into the other, and it is a distributed database: no single machine holds it all.
- The **DNS is itself an application running** over the network, and we will open it shortly.

Application Layer

Transport-service requirements of applications

- Different network applications have different requirements from the transport service:
  - **Reliability** of data transfer: data sent by one end of the application is delivered correctly and completely to the other end.
  - **Throughput**: the rate at which the sending process can deliver bits to the receiving process (bit/sec).
  - **Timing**: every bit that the sender pumps into the socket arrives at the receiver's socket within a time range.
  - **Security**: confidentiality (only authorized parties can read the data), integrity, authentication, when required
    - Security is typically provided by additional protocols, such as TLS (Transport Layer Security), rather than by TCP or UDP themselves.
    - TLS operates above the Transport layer and is usually considered part of the Application layer
    - E.g., HTTP → TLS → TCP → IP
    - The application encrypts before the socket and decrypts after it.

Application Layer

Transport-service requirements of applications

- Standard Internet transport protocols provide only limited support for these application requirements.
  - **Reliability** provided by TCP, not by UDP.
  - **Throughput**: no guaranteed minimum throughput from TCP or UDP.
  - **Timing**: no delivery-delay guarantee from TCP or UDP.
  - **Security**: not provided directly by TCP or UDP; typically added by protocols such as TLS.

Application Layer

What real applications need

- Application needs

Application	Data loss	Throughput	Delay sensitivity
File transfer	No loss	Elastic	Low
E-mail	No loss	Elastic	Low
Web	No loss	Elastic / variable	Moderate
Internet telephony	Some loss tolerated	Sustained rate	High
Video conferencing	Some loss tolerated	Moderate / high	High
Stored audio/video	Some loss tolerated	Sustained rate	Moderate
Interactive games	Some loss tolerated	Usually modest	High

Elastic means the application uses whatever throughput it finds: more is better, less still works.

Application Layer
TCP & UDP

The two main transport protocols used in the Internet are TCP and UDP

- **TCP** (Transmission Control Protocol): connection-oriented transport protocol; establishes a connection through a handshake and provides reliable, ordered byte-stream delivery
  - TCP also adapts its sending rate in response to network congestion.
- **UDP** (User Datagram Protocol): connectionless transport protocol; provides a lightweight datagram service with no connection setup and no guarantee of delivery or ordering.
  - UDP itself provides no flow or congestion control.
  - UDP deliberately provides a minimal transport service, leaving additional mechanisms to the application.

Application Layer
Application Protocols

- An **application-layer protocol** defines how network application's processes, running on different end systems, pass messages to each other.
  - How are messages structured? What are the meanings of the various fields in the messages? When do the processes send the messages?

- Specifically, application-layer protocols define:
  1. The **types** of messages exchanged (for example request messages and response messages).
  2. The **syntax** of the various message types, such as the fields in the message and how the fields are delineated.
  3. The **semantics** of the fields, i.e., the meaning of the information in the fields.
  4. The **rules** for determining when and how a process sends messages and responds to messages.

Application Layer

Application Protocols

The Web is an application; HTTP is one component of it.

- The Web also relies on document and data formats such as HTML, and on software such as browsers and Web servers.
- HTTP defines the messages exchanged between clients and servers: their types, syntax, semantics, and exchange rules.
- Similarly, a video-on-demand application includes clients, servers and media formats, and may use protocols/standards such as HTTP and DASH (Dynamic Adaptive Streaming over HTTP).

Why insist on the distinction?

Communicating components must agree on the protocols and data formats needed for interoperability; their internal implementations can be completely different.

Application Layer

Some Application Protocols

- There are several application-layer protocols that are commonly used on networks (and internet), here some example.

Application	Description
DHCP	Dynamic Host Configuration Protocol, assigns IP addresses
DNS	Domain Name System, translate website names to IP addresses
HTTP/HTTPS	HyperText Transfer Protocol (Secure), transfer web pages
SMTP/SMTPS	Simple Mail Transfer Protocol (Secure), sends email messages
SNMP	Simple Network Management Protocol, manages network devices
Telnet/SSH	Teletype Network (Secure SHell), allows command-line interfacing with remote hosts
FTP/FTPS	File Transfer Protocol (Secure), used to transfer files

Application Layer

Some Application Protocols

- Some application may exchange **sensitive information** (e.g., user's credentials, personal data, etc.).
- On public networks, messages should be **secured**.

Application	Description
DHCP	Dynamic Host Configuration Protocol, assigns IP addresses
DNS	Domain Name System, translate website names to IP addresses
HTTP/HTTPS	HyperText Transfer Protocol (Secure), transfer web pages
SMTP/SMTPS	Simple Mail Transfer Protocol (Secure), sends email messages
SNMP	Simple Network Management Protocol, manages network devices
Telnet/SSH	Teletype Network (Secure SHell), allows command-line interfacing with remote hosts
FTP/FTPS	File Transfer Protocol (Secure), used to transfer files

Application Layer
FTP and FTPS

- **FTP** (File Transfer Protocol) is one of the **oldest** protocols defined in Internet (first version in 1971) and is used to transfer files between hosts over network.

- In the basic version of FTP data transfer is in **clear-text** (username, password, and files) so it is best used in local or private applications.

- The **secure** version FTPS (FTP Secure) protects username and password and encrypt contents.

- Both FTP and FTPS have two components:
  - The **protocol** that specifies commands (show, get, delete files, etc.).
  - A **software application** implementing the protocol (client-side and a server-side software).

Application Layer
FTP and FTPS

- Unlike many application protocols, FTP does not carry commands and data over the same connection.
  - The **control connection**, to server port 21, is established first and remains open for the session: commands and replies only.
  - Each file transfer or directory listing uses a **separate data connection**, which is closed when the transfer ends.
  - Keeping control information separate from data is called out-of-band control.

- Who opens the data connection?
  - In **active mode**, the server opens it back (from port 20 to client-selected port) to the client, which may be problematic with NATs (Network Address Translation) or firewalls.
  - In **passive mode**, server waits, client opens the data connection server-selected port.

Application Layer
FTP Example

- In Linux we can use ftp and vsftpd (very secure FTP daemon) as client and server applications respectively.

- On the server machine:
  - Install vsftpd:
    - $ sudo apt-get install vsftpd
  - Generate the user
    - $ sudo adduser ftp_user
    - $ sudo -iu ftp_user (per entrare come nuovo user)
  - Check ftp server running
    - $ service vsftpd status (check if daemon is running)

- On the client machine:
  - Install ftp:
    - $ sudo apt-get install ftp (it is already available in Ubuntu)
  - Connect to server:
    - $ ftp ADDRESS (usr and pass will be asked)
  - Close connection:
    - $ exit

Note: The server side of this application is implemented as a **daemon**: a program that runs in background and waits for clients to connects. This is a common approach.

Application Layer
FTP Example

- In Linux we can use ftp and vsftpd (very secure FTP daemon) as client and server applications respectively.

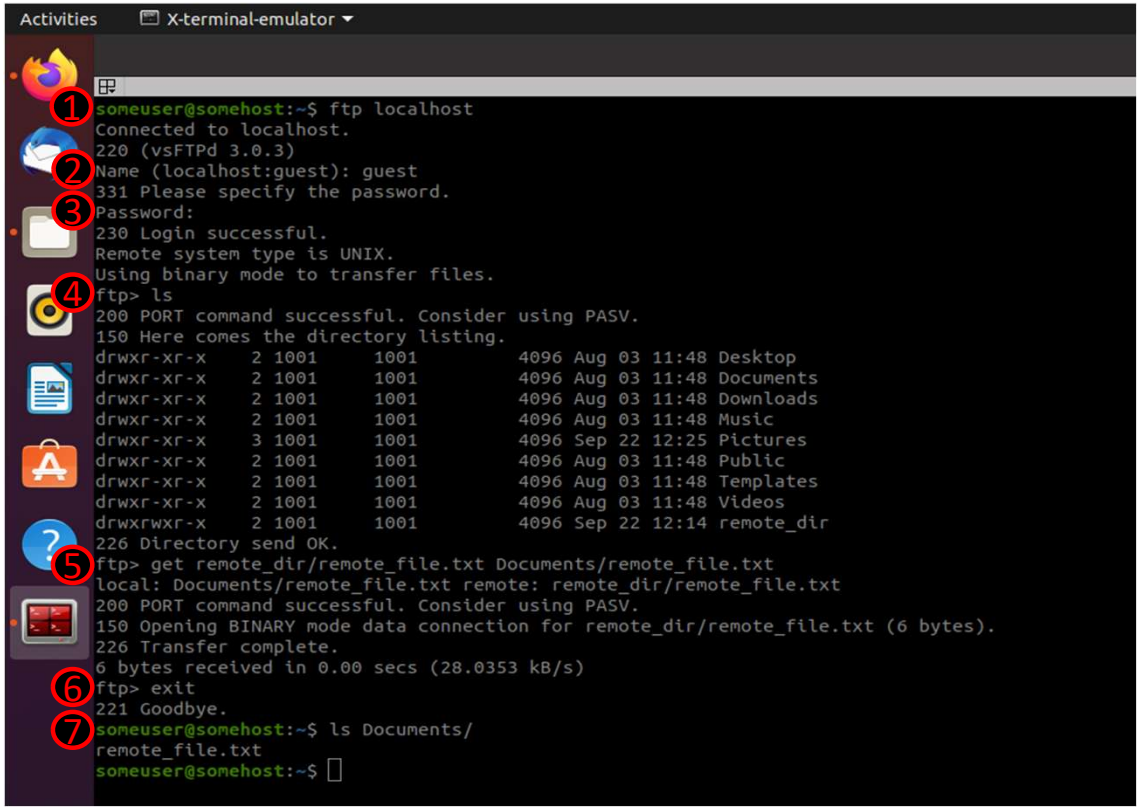
```
$ sudo apt install vsftpd              server side, once
$ systemctl status vsftpd              is the daemon running?

$ ftp test.rebex.net                   client side
Name: demo
Password: password
230 Login successful.
ftp> ls                                from here on, the ftp> prompt
ftp> get report.pdf                    server to client
226 Transfer complete.
ftp> bye
```

- Nearly every server you will write in this course has this shape: a daemon, waiting. FTP sends the password in clear: today files travel over SFTP, inside SSH, or over HTTPS.

Application Layer

FTP Example (get a file)



Here we connect to ourselves (localhost), but an IP address can be specified:

1. Run ftp with the address of the remote host.
2. Insert username with which you want to log on remote host.
3. Insert password
4. Run ls to see files and directories
5. Copy the file remote_dir/remote_file.txt into (local) Documents
6. Exit from ftp
7. Check the file

Application Layer
FTP Example

- In Linux we can use ftp and vsftpd (very secure FTP daemon) as client and server applications respectively.

- New terminal
  - Check the server in ascolto: ss –ltn | grep ':21'
    - LISTEN 0 32 *:21 *:*
  - Check di client e server: ss –tn | grep ':21'
    - ESTAB 0 0 127.0.0.1:39256 127.0.0.1:21 client
    - ESTAB 0 0 [::ffff:127.0.0.1]:21 [::ffff:127.0.0.1]:39256 server

- Prompt ftp
  - ftp> status try passive first, then active mode

Application Layer

FTP Common Commands

Common FTP commands:

help - List all available FTP commands.

cd - Change directory on remote machine.

lcd - Change directory on local machine.

ls - View the names of the files and directories in the current remote directory.

mkdir - Create a new directory within the remote directory.

pwd - Print the current working directory on the remote machine.

delete - Delete a file in the current remote directory.

rmdir- Remove a directory in the current remote directory.

get - Copies a file from the remote server to the local machine.

put - Copies a file from the local machine to the remote machine.

Application Layer
FTP Common Commands

Common FTP commands:

To abilitate put, mkdir, etc

Open vsftpd config file:

sudo nano /etc/vsftpd.conf

Change:

write_enable=YES

Restart:

sudo systemctl restart vsftpd

Application Layer
FTP Commands and Protocol

FTP commands use the FTP protocol:

User command
↓
FTP client
↓
FTP protocol messages
↓
FTP server

ftp> ls

EPSV the client sends EPSV (Extended Passive Mode). The server replies with a port number, and the client opens the data connection to the server.
LIST

EPRT the client sends EPRT (Extended PORT), specifying its address and port. The server opens the data connection to the client.
LIST

Summary

A network application consists of programs running on end systems; application logic does not normally run in the network core.

- The developer picks an architecture — client-server, P2P, or a hybrid architecture.
- What communicates are processes, through sockets. In a client-server architecture, the client initiates communication with the server.
- Reaching a process requires an IP address for the host and a port number for the application endpoint: one from the Network layer, one from the Transport layer.
- The Transport layer provides different services depending on the protocol. TCP provides reliable delivery; the Internet transport service does not normally guarantee throughput or timing. Security is typically provided by additional protocols.
- An application-layer protocol defines the types of messages exchanged, their syntax and semantics, and the rules governing their exchange. It is only one component of a network application.